Exercise2

January 23, 2026

1 Exercise 2:

2.1-

[1]:
```
# numpy.random.randn
# importing numpy
import numpy as np

# setting the parameters
num_samples = 100
D = 10

# generating random directions
directions = np.random.randn(num_samples, D)

# testing if the rows correspond to unit vectors
print(f"Norm of the first vector: {np.linalg.norm(directions[0])}")
print(f"First direction vector: {directions[0]}")
```

```
Norm of the first vector: 3.64917693968668
First direction vector: [ 0.30827857  1.4341027  -0.95602814 -2.29254852
0.7958618   0.16178678
  0.57471757 -1.14087136  1.23160452  1.08937099]
```

2.1.1- The element of "directions" are random numbers generated from the normal distribution.

2.1.2- Based on the norm of first vector, the rows generated are not unit vectors.

[6]:
```
product = directions * directions
print(f"First row of the product matrix: {product[0]}")
```

```
First row of the product matrix: [7.24869558e-02 1.20313388e+00 1.86612470e+00
5.73770451e+00
 8.62010997e-01 2.73712999e-03 2.99839183e-01 3.72417955e-03
 3.55366426e+00 2.97837405e+00]
```

the operation direction * direction results in the element-wise product of each row with itself

[10]:
```
print(f"Total sum of squares : {np.sum(product)} \n") # results in total sum of␣
 ↪squares of all elements
```

```
print(f"sum of squares for each column: {np.sum(product,axis=0)} \n") # results␣
  ↪in sum of squares for each column
psum = np.sum(product,axis=1) # results in sum of squares for each row
print(f"sum of squares for each row: {psum}")
```

```
Total sum of squares : 1010.3200772549991

sum of squares for each column: [115.63278731  99.12709577 131.7248395
115.29423577  92.64512247
  83.47308685  94.52687778 107.07719701  84.30413656  86.51469823]

sum of squares for each row: [16.57979985 12.15351171 18.1150835  11.51877927
5.62158219 15.7728088
  8.49643136  8.48756564 14.37840439  3.17038213 14.12794974 19.53446768
 26.94045311  6.76604752 13.71411537  2.74312676  6.91346045  8.10772315
 11.8541766   8.57632435 13.05624003 13.32124836 19.83460918  4.50608779
  6.73733025  5.63913128  4.98216711  6.1486342   3.77662725 14.28520761
 24.51342024  7.27644947  5.503828    7.78836851  9.02531066 18.29154255
 10.9657246  12.93851257 19.15019699  9.60590799  9.76002244 14.76300735
 10.4884511  11.33827082  9.30837487  8.16174099  8.55773131  5.88390663
 11.80578861 13.45151911  6.10635315 10.71337551  6.71074336 13.40950139
  9.24650134  5.58182988  9.59119292  5.62929499  7.50102019  5.90317736
  3.59147285  3.93649746 19.45654932  9.49601775 15.69058709  4.19377624
 14.50018527 10.36766429 12.39909283  9.34052387  7.78979087 18.57330283
  8.45545103 18.41018514 10.15908741  9.12063966  7.55022646  5.19641727
 13.22636657  7.1828445  13.81210116  7.53662366 12.60007578  3.59389728
  6.01532223  6.92919635  6.38218916  6.52267937  9.68800462  8.48206561
  5.0366721   4.91661902  5.46738803  3.53495078  6.18026089 10.50977496
 14.41365487  5.83559561 16.85048363 12.54530192]
```

2.3-

[11]:
```
# computing the norms and normalizing the directions
norms = np.sqrt(psum)[:,np.newaxis] # norms = np.sqrt(psum) does not work due␣
  ↪to broadcasting issues
directions = directions/norms
```

[12]:
```
# Checking if the rows correspond to unit vectors
print(f"Norms vectors after normalization: {np.linalg.norm(directions,␣
  ↪axis=1)}")
```

```
Norms vectors after normalization: [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1.]
```